

Gestor de Tareas (To-Do List)

Josué Román J.

Introducción a los Repositorios de Código Distribuido

Profesor de Cátedra: Felipe Gómez

<https://github.com/Josusu682/to-do>

5 de julio de 2026

Índice

Descripción del proyecto	3
Estructura del repositorio	3
Evolución del desarrollo.....	4
Análisis del historial de Git.....	4
Análisis de integración de cambios.....	6
Visualización del repositorio	7
Análisis de métricas	7
Reflexión final	7

Descripción del proyecto

Problema abordado: La necesidad de gestionar tareas diarias y mantener el enfoque mediante una herramienta rápida, sin depender de aplicaciones pesadas, creación de cuentas o conexiones permanentes a bases de datos en la nube.

Objetivo: Desarrollar una aplicación web ligera (frontend-only) para la gestión personal de tareas, utilizando herramientas de control de versiones y asegurando la persistencia de los datos en el navegador del usuario de forma local.

Funcionalidades principales:

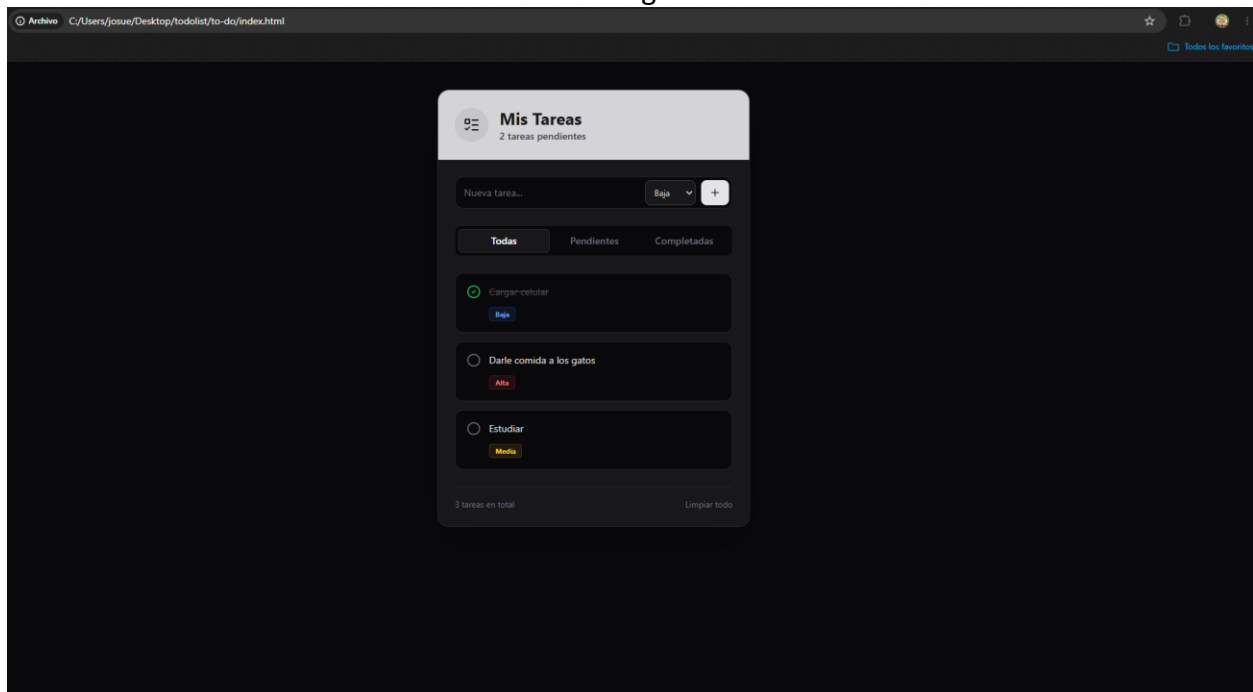
Creación de nuevas tareas dinámicas.

Cambio de estado visual (marcar tareas como completadas).

Eliminación individual y limpieza total de la lista.

Sistema de filtros de visualización (Todas, Pendientes, Completadas).

Persistencia de información mediante LocalStorage.



Estructura del repositorio

El proyecto fue construido bajo un enfoque minimalista para mantener un control de versiones ordenado, compuesto por los siguientes archivos clave:

- index.html: Archivo principal o "hotspot" del proyecto. Contiene la estructura visual semántica, los estilos aplicados mediante Tailwind CSS (vía CDN) y la lógica de estado y reactividad programada en Vue.js.
- README.md: Documento de especificaciones del proyecto. Detalla el contexto académico, el *stack* tecnológico utilizado y las instrucciones precisas para la ejecución local.
- .gitignore: Archivo de configuración de Git, preparado para excluir potenciales directorios de dependencias (node_modules), variables de entorno y archivos autogenerados por el sistema operativo.
- Informe_COM4602.pdf: El presente documento que expone el análisis del desarrollo y del repositorio.

Evolución del desarrollo

El proyecto se construyó de manera iterativa, registrando actividad en el historial de Git a lo largo del periodo de desarrollo establecido:

- **Día 1:** Inicialización del repositorio local y creación de la estructura base (README.md y .gitignore).
- **Día 2:** Creación de la rama desarrollo-ui e implementación del primer prototipo visual estático de la lista de tareas.
- **Día 3:** Pruebas de integración en main forzando una colisión controlada en el código para comprender la mecánica de Git frente a discrepancias.
- **Día 4:** Apertura de la rama desarrollo-logica, persistencia de datos mediante la API LocalStorage.
- **Día 5:** Implementación del rediseño oscuro (Dark Mode), integración de iconografía vectorial (Lucide) y el sistema de filtros en la rama de interfaz gráfica, a la par de la actualización de la documentación en la rama principal.
- **Día 6:** Integración final de las ramas, resolución manual del conflicto estructural definitivo derivado del desarrollo paralelo, y compilación del informe final de métricas.

Análisis del historial de Git

Número total de commits: 17

Días con actividad: 6 días diferentes

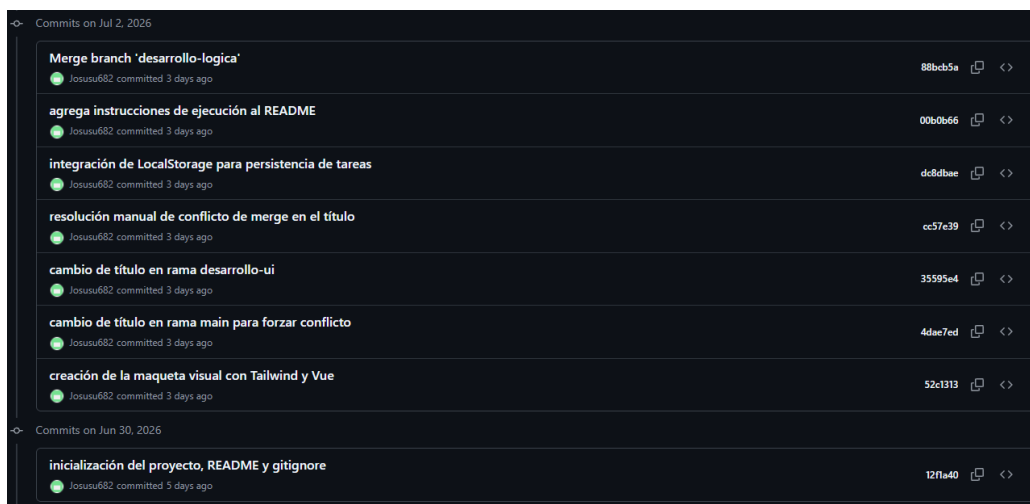
Archivos más modificados: El archivo index.html fue el principal foco de iteraciones, seguido por README.md.

Commits más relevantes:

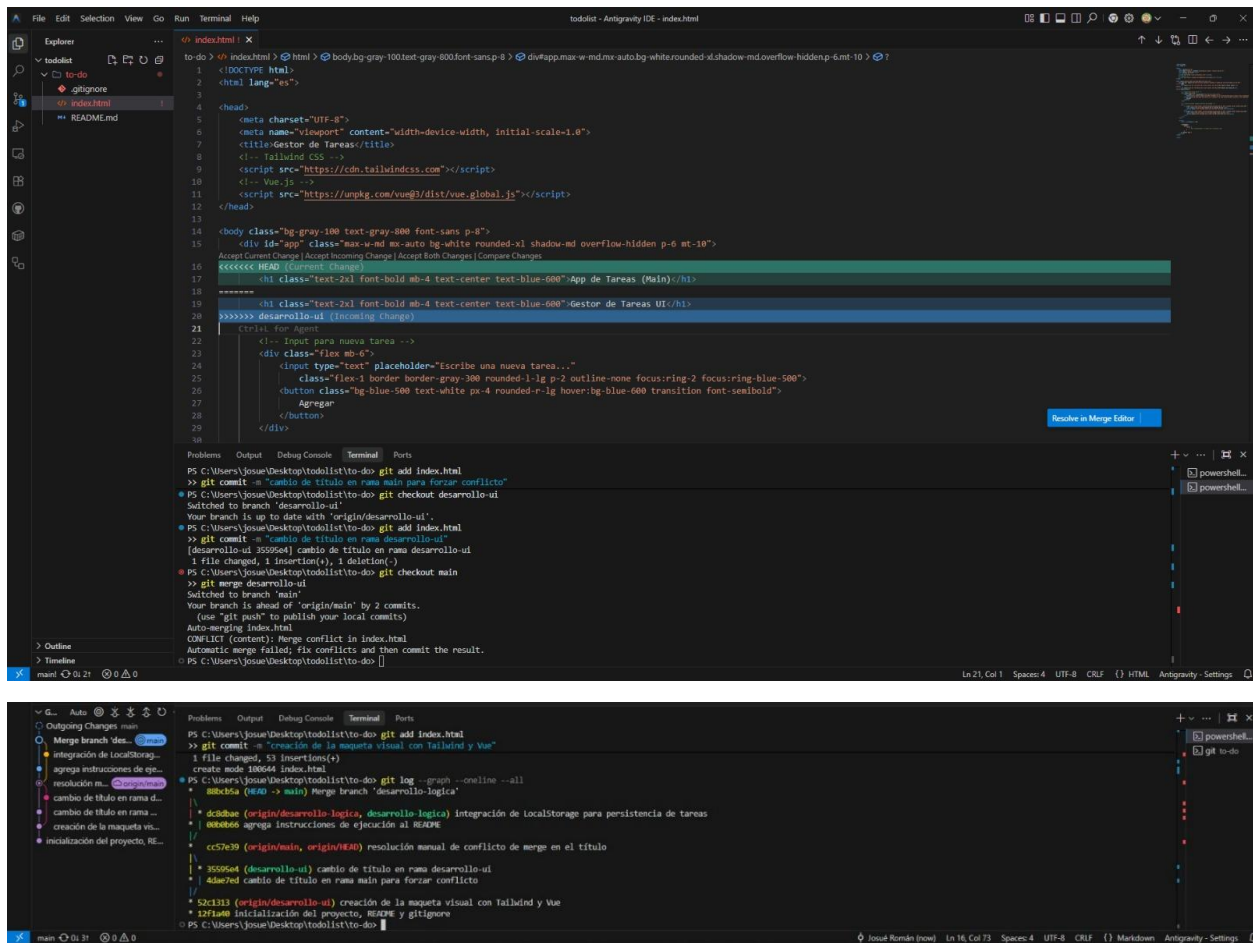
- integración de LocalStorage para persistencia de tareas (Aportó el mayor nivel de funcionalidad).
- resolución manual de conflicto de merge en el título (Demuestra el control ante choques de código).
- rediseño oscuro, integración de iconos y lógica de filtrado de tareas (Cambio estructural de la interfaz de usuario).

Ramas utilizadas: main (como línea base de producción), desarrollo-ui (aislamiento de características gráficas) y desarrollo-logica (aislamiento de funcionalidades lógicas).

```
PS C:\Users\josue\Desktop\todolist\to-do> git log --pretty=format:"%h | %ad | %an | %s" --date=short
a1ac59d | 2026-07-05 | Josué Román | agrega ideas al README
694470a | 2026-07-05 | Josué Román | añade sección de mejoras futuras y traducción al inglés
342771c | 2026-07-05 | Josué Román | corrige renderizado de iconos y añade color verde al completar tarea
fcb3083 | 2026-07-05 | Josué Román | mejora en la redacción de las instrucciones de ejecución
1818551 | 2026-07-05 | Josué Román | añade edición en línea de tareas y sistema de etiquetas de prioridad
8e39ca6 | 2026-07-05 | Josué Román | Merge branch 'desarrollo-ui'
a47ce60 | 2026-07-04 | Josué Román | actualización detallada del README con stack tecnológico e instrucciones
eeb78d1 | 2026-07-04 | Josué Román | rediseño oscuro, integración de iconos y lógica de filtrado de tareas
88bcb5a | 2026-07-02 | Josué Román | Merge branch 'desarrollo-logica'
00b0b66 | 2026-07-02 | Josué Román | agrega instrucciones de ejecución al README
dc8dbae | 2026-07-02 | Josué Román | integración de LocalStorage para persistencia de tareas
cc57e39 | 2026-07-02 | Josué Román | resolución manual de conflicto de merge en el título
35595e4 | 2026-07-02 | Josué Román | cambio de título en rama desarrollo-ui
4dae7ed | 2026-07-02 | Josué Román | cambio de título en rama main para forzar conflicto
52c1313 | 2026-07-02 | Josué Román | creación de la maqueta visual con Tailwind y Vue
12f1a40 | 2026-06-30 | Josué Román | inicialización del proyecto, README y gitignore
PS C:\Users\josue\Desktop\todolist\to-do>
```



Merges realizados: Se realizaron integraciones de tipo avance rápido (Fast-forward) y al menos un *three-way merge*.



Evidencia y explicación de la resolución manual de conflictos: Durante el desarrollo, ambas ramas intentaron modificar simultáneamente la misma sección del documento principal (index.html). Git inyectó marcas de conflicto (como <<<<<<< HEAD). La resolución manual consistió en editar el archivo, eliminar las marcas del gestor y conservar deliberadamente la versión definitiva antes de confirmar la fusión.

Análisis de integración de cambios

Descripción de las ramas utilizadas y objetivos:

- **main:** Rama de producción encargada de mantener el código estable y funcional, además de albergar la documentación del repositorio.
- **desarrollo-ui:** Creada con el objetivo estricto de iterar sobre el aspecto visual (maquetación, colores, estilos de Tailwind) sin comprometer la lógica en caso de errores en la sintaxis.
- **desarrollo-logica:** Enfocada en construir el ciclo de vida de la aplicación con Vue.js, manejar el DOM y asegurar el almacenamiento.

Captura del historial mostrando el merge:

```
>> git merge desarrollo-uitodolist\to-do>
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\josue\Desktop\todolist\to-do> git commit -m "añade edición en línea de tareas y sistema de etiquetas de prioridad"
```

Explicación del conflicto generado: El conflicto estructural más crítico surgió cuando main avanzó con la configuración base de guardado local, mientras desarrollo-ui reemplazó toda la estructura HTML para aplicar un rediseño completo (modo oscuro). Al intentar fusionar, Git detectó modificaciones incompatibles en casi la totalidad del archivo index.html.

Explicación de la resolución manual aplicada: Para resolver la discrepancia sin romper la aplicación, se abrieron los archivos en el editor de código, descartando el código obsoleto de la rama main y aceptando íntegramente los cambios entrantes de desarrollo-ui (que ya traían adaptada la lógica de almacenamiento), limpiando las marcas de Git y registrando el commit respectivo.

Visualización del repositorio

```
* c3b5235 (HEAD -> main, origin/main, origin/HEAD) configura gitignore para excluir secretos de entorno y apuntes locales
* a1ac59d agrega ideas al README
* 694470a añade sección de mejoras futuras y traducción al inglés
* 342771c corrige renderizado de iconos y añade color verde al completar tarea
* fcb3083 mejora en la redacción de las instrucciones de ejecución
* 1818551 añade edición en línea de tareas y sistema de etiquetas de prioridad
* 8e39ca6 Merge branch 'desarrollo-ui'
| \
| * eeb78d1 (origin/desarrollo-ui, desarrollo-ui) rediseño oscuro, integración de iconos y lógica de filtrado de tareas
| * | a47ce60 actualización detallada del README con stack tecnológico e instrucciones
| * | 88bcb5a Merge branch 'desarrollo-logica'
| \ \
| * | dc8dbae (origin/desarrollo-logica, desarrollo-logica) integración de LocalStorage para persistencia de tareas
| * | 00b0b66 agrega instrucciones de ejecución al README
| / /
| * | cc57e39 resolución manual de conflicto de merge en el título
| \ \
| * 35595e4 cambio de título en rama desarrollo-ui
| * 4dae7ed cambio de título en rama main para forzar conflicto
| /
* 52c1313 creación de la maqueta visual con Tailwind y Vue
* 12f1a40 inicialización del proyecto, README y gitignore
```

Análisis de métricas

Al interpretar los gráficos del repositorio, se evidencia una clara estrategia de *Separación de Preocupaciones* (Separation of Concerns). La presencia de múltiples ramificaciones confirma que el desarrollo no fue lineal; las *features* fueron aisladas en sus propias líneas de tiempo para garantizar estabilidad. El "hotspot" (punto caliente de modificaciones) fue indiscutiblemente index.html. Al tratarse de una arquitectura de componentes en un solo archivo (adaptación de Single-File Component mediante CDN), este archivo sufrió constantes inserciones y eliminaciones, reflejando iteraciones progresivas en lugar de ser subido de golpe.

Reflexión final

El desarrollo de este proyecto comprobó en la práctica que Git es mucho más que una herramienta para subir código a la nube; es un seguro de vida para el desarrollador. A través

del uso metódico de *commits* atómicos, se comprobó la utilidad de documentar el "por qué" de cada cambio. El manejo de *ramas* demostró cómo se pueden explorar nuevas funcionalidades o rediseños completos sin el temor a destruir el código funcional de la aplicación. Asimismo, la experiencia resolviendo colisiones demostró que un *conflicto de merge* no es un error destructivo del sistema, sino un mecanismo de protección indispensable en el desarrollo de software moderno. Git detiene el proceso para forzar la intervención humana, asegurando que ninguna línea de código valiosa se sobrescriba de manera silenciosa o accidental, un pilar fundamental en cualquier flujo de trabajo, ya sea individual o distribuido en equipo.